`

AMRITA VISHWA VIDYAPEETHAM

AHEAD Online

Foundations of Computer Systems

Objectives

- Process Management.

What is a process?

- A process is a program in execution.

A process includes,

- Program code, the current activity, as represented by the value of the program counter and the contents of the processor's registers.
- Process stack, which contains temporary data (such as function parameters, return addresses, and local variables), and a data section, which contains global variables.
- A heap, which is memory that is dynamically allocated during process run time.

Program vs Process

- Program is *passive* entity stored on disk.
- Process is *active entity.*
- Program becomes process when executable file loaded into memory.
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes.

Process Management

The operating system is responsible for the following activities in connection with process management:

- Scheduling processes and threads on the CPUs.
- Creating and deleting both user and system processes.
- Suspending and resuming processes.
- Providing mechanisms for process synchronization.
- Providing mechanisms for process communication.

Process States

- A Process may be in any one of the following Process States at any given instant:
  - New
  - Running
  - Waiting
  - Ready
  - Terminated

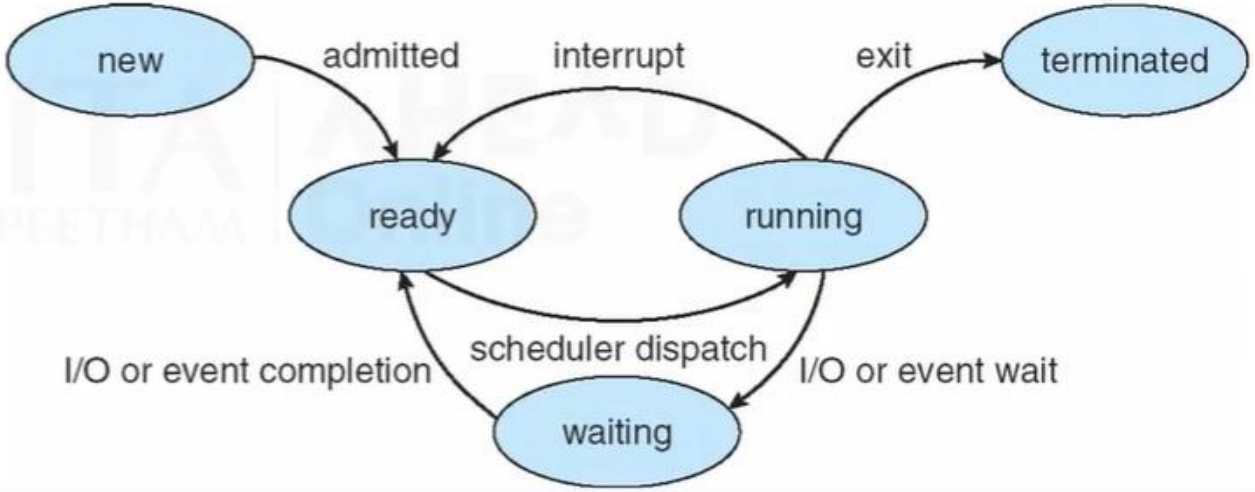


Image Courtesy:Silberschatz, Galvin and Gagne – 'Operating System Concepts' –

Process Control Block

- Data structure to store all the information about each individual process
- When a process is created (initialized or installed), the operating system creates a corresponding process control block.
- Information in a process control block is updated during the transition of process states.
- When the process terminates, its PCB is returned to the pool from which new PCBs are drawn.

Process Control Block

Image Courtesy:Silberschatz, Galvin and Gagne – 'Operating System Concepts' –

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Process Switching

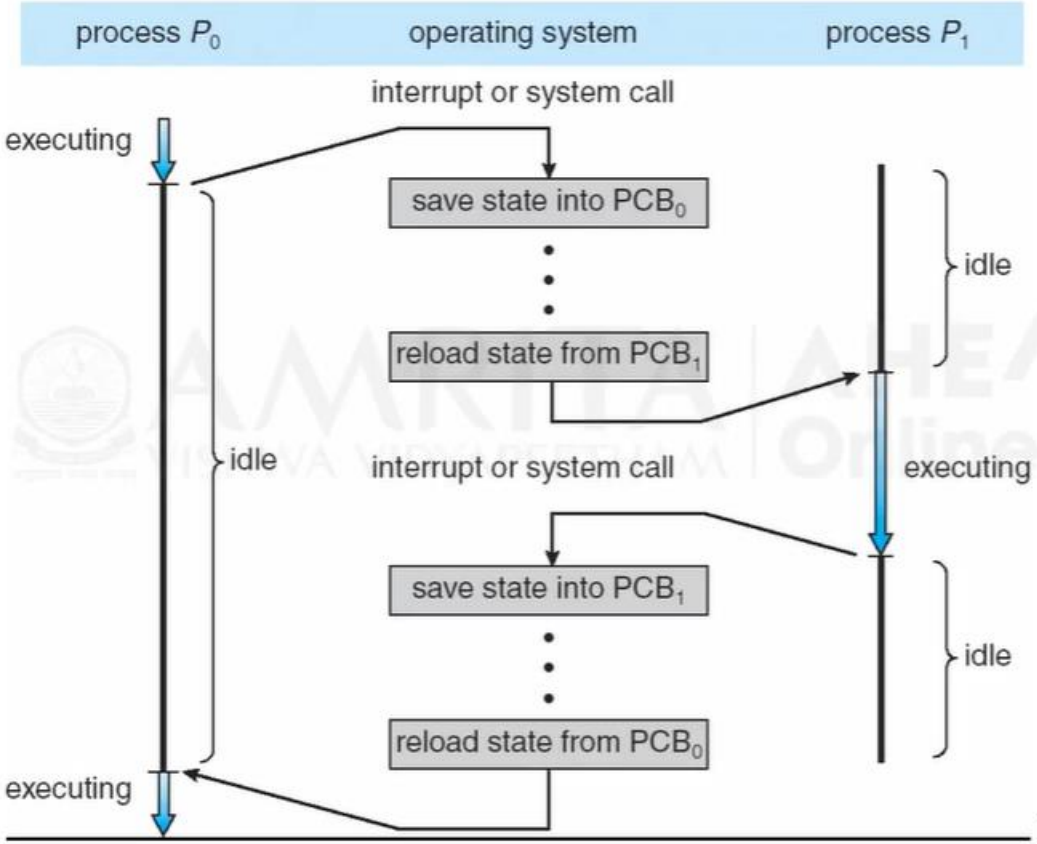


Image Courtesy:Silberschatz, Galvin and Gagne – 'Operating System Concepts' – 9th Edition.

Process Management

Threads

- A thread is a basic unit of CPU utilization.
- It is having
    - A thread ID
    - Program counter
    - Register set
    - Stack
- Threads shares code section, data section, and other operating-system resources, such as open files and signals.

Process Management

Concurrency

- Several processes are running concurrently—during overlapping time periods—instead of sequentially.
- One completing before the next starts.
- Concurrency leads to issues of resource sharing and maintaining the integrity of shared resources.

Process Management

Concurrency –Critical Section Problem

- In concurrent programming part of the program where a shared resource is accessed is the critical section or critical region.
- The critical section cannot be executed by more than one process at a time.

Process Management

Concurrency –Deadlocks

- When several processes are executing concurrently, they may compete for a finite number of resources..
- If the requested resource is not available process enters in to waiting state.
- When waiting processes are never again able to change state, because the resources one of them has requested is continuously held by the other and vice versa. This circular wait situation is called a deadlock
- Deadlock handling is a key feature in Operating Systems.

Takeaways

- What is a process?
- Program vs Process.
- Process States.
- Process Control Block.
- Threads.
- Concurrency.
- Deadlock.

References

1 Fundamentals Computers, V Rajaraman, Neeharika Adabala, Phi Learning Pvt. Ltd

2 J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelth Edition, 2014

3 Computer Fundamentals, Anita Goel, Pearson

Objectives

- Process Management.

Process Management

The operating system is responsible for the following activities in connection with process management:

- Scheduling processes and threads on the CPUs.
- Creating and deleting both user and system processes.
- Suspending and resuming processes.
- Providing mechanisms for process synchronization.
- Providing mechanisms for process communication.

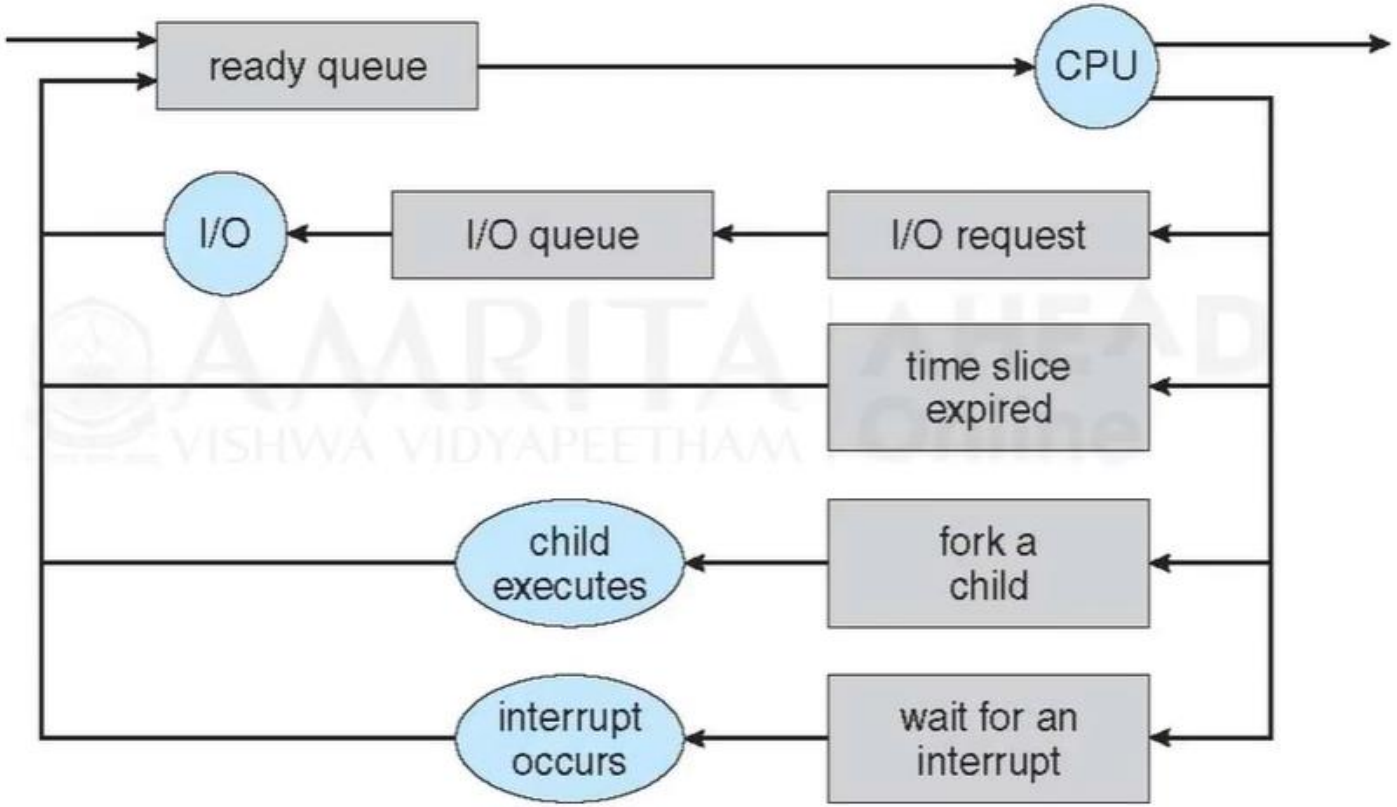
Process Scheduling

- In multiprogramming system to maximize CPU utilization some processes will be running all the time.
- Process scheduler will select the available process for execution on CPU.
- Process scheduler maintains a scheduling queue for processes.

Scheduling queues

- **Job queue** –set of all processes in the system.
- **Ready queue** –set of all processes residing in main memory, ready and waiting to execute.
- **Device queues** –set of processes waiting for an I/O device
- Processes migrate among the various queues.

Queueing Diagram



Schedulers

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts

Schedulers

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
  - Remove process from memory, store on disk, bring back in from disk to continue execution(**swapping).**

Process Operations

- **Process Creation**
  - **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
  - Process identified and managed via a **process identifier** (**pid**)
  - **fork()** system call creates new process in unix.
- **Process Termination**
  - Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
  - Returns status data from child to parent (via **wait()**)

Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch.**
- **Context** of a process represented in the PCB.
- Context-switch time is overhead; the system does no useful work while switching.

Inter process Communication

- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

Process Synchronization

- Message passing may be either blocking or non-blocking
- **Blocking** is considered **synchronous**
  - **Blocking send** - the sender is blocked until the message is received
  - **Blocking receive** - the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** - the sender sends the message and continue
  - **Non-blocking receive** - the receiver receives a valid or null message.

Takeaways

- Process management by OS.
- Process scheduling.
- Scheduling queues.
- Operations of processes.
- Context switch.
- Inter process communication.
- Process synchronization.

References

1 Fundamentals Computers, V Rajaraman, Neeharika Adabala, Phi Learning Pvt. Ltd

2 J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelth Edition, 2014

3 Computer Fundamentals, Anita Goel, Pearson

Objectives

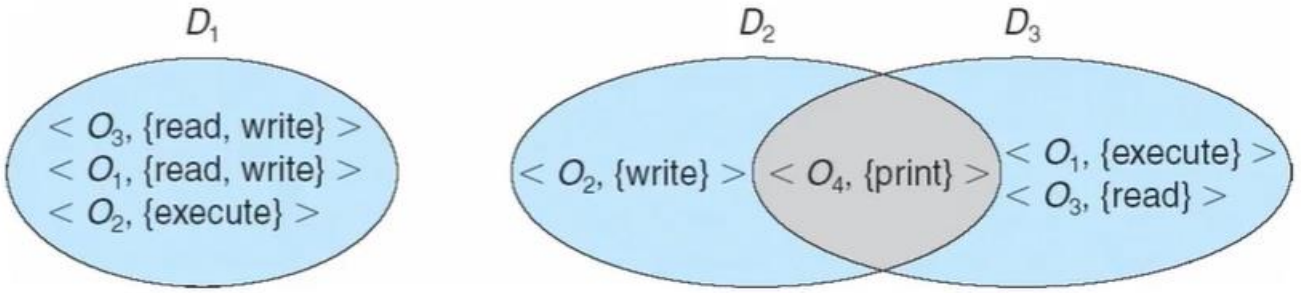
- Security and Protection.

Goals of Protection

- Guiding principle –principle of least privilege
  - Programs, users and systems should be given just enough privileges to perform their tasks.
  - An operating system following the principle of least privilege implements its features, programs, system calls, and data structures so that failure or compromise of a component does the minimum damage and allows the minimum damage to be done.

Protection Domain

- Protection domain of a process which specifies the resources that the process may access.
- Access-right = <*object-name*, *rights-set*>
  where *rights-set* is a subset of all valid operations that can be performed on the object.
- Domain = set of access-rights



Access Matrix

- General Model for protection
- View protection as a matrix (*access matrix*)
- Rows represent domains
- Columns represent objects
- *Access(i, j)* is the set of operations that a process executing in $Domain_i$ can invoke on $Object_j$

Access Matrix

domain \ object	F_1	F_2	F_3	printer
D_1	read		read	
D_2				print
D_3		read	execute	
D_4	read write		read write	

Security Problem

- Security must consider external environment of the system, and protect the system resources
- Intruders (crackers) attempt to breach security
- Threat is potential security violation
- Attack is attempt to breach security
- Attack can be accidental or malicious
- Easier to protect against accidental than malicious misuse

Security Violations

- Categories
  - Breach of confidentiality
  - Breach of integrity
  - Breach of availability
  - Theft of service
  - Denial of service
- Methods
  - Masquerading (breach authentication)
  - Replay attack
    - Message modification
  - Man-in-the-middle attack
  - Session hijacking

Program Threats

- **Trojan Horse**
  - Code segment that misuses its environment
  - Exploits mechanisms for allowing programs written by users to be executed by other users
- **Trap Door**
  - Specific user identifier or password that circumvents normal security procedures
  - Could be included in a compiler
- **Logic Bomb**
  - Program that initiates a security incident under certain circumstances
- **Stack** and **Buffer Overflow**
  - Exploits a bug in a program (overflow either the stack or memory buffers)

System and Network Threats

- **Worms** – use spawn mechanism; standalone program
- Internet worm
  - Exploited UNIX networking features (remote access) and bugs in *finger* and *sendmail* programs
  - Grappling hook program uploaded main worm program
- **Port scanning**
  - Automated attempt to connect to a range of ports on one or a range of IP addresses
- **Denial of Service**
  - Overload the targeted computer preventing it from doing any useful work
  - Distributed denial-of-service (DDOS) come from multiple sites at once

Takeaways

- Principle of Protection.
- Protection Domain.
- Access Matrix.
- Security Problem.
- Security Violations.
- Major Threats.

References

1 Fundamentals Computers, V Rajaraman, Neeharika Adabala, Phi Learning Pvt. Ltd

2 J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelth Edition, 2014

3 Computer Fundamentals, Anita Goel, Pearson

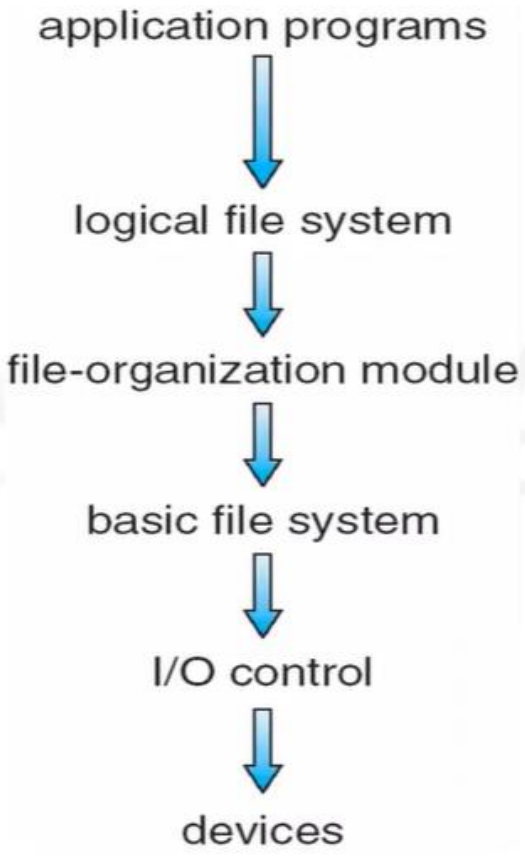
Objectives

- File Management.

File System Structure

- File structure
  - Logical storage unit
  - Collection of related information
- **File system** resides on secondary storage (disks)
  - Provided user interface to storage, mapping logical to physical
  - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- **File control block** –storage structure consisting of information about a file

Layered File System



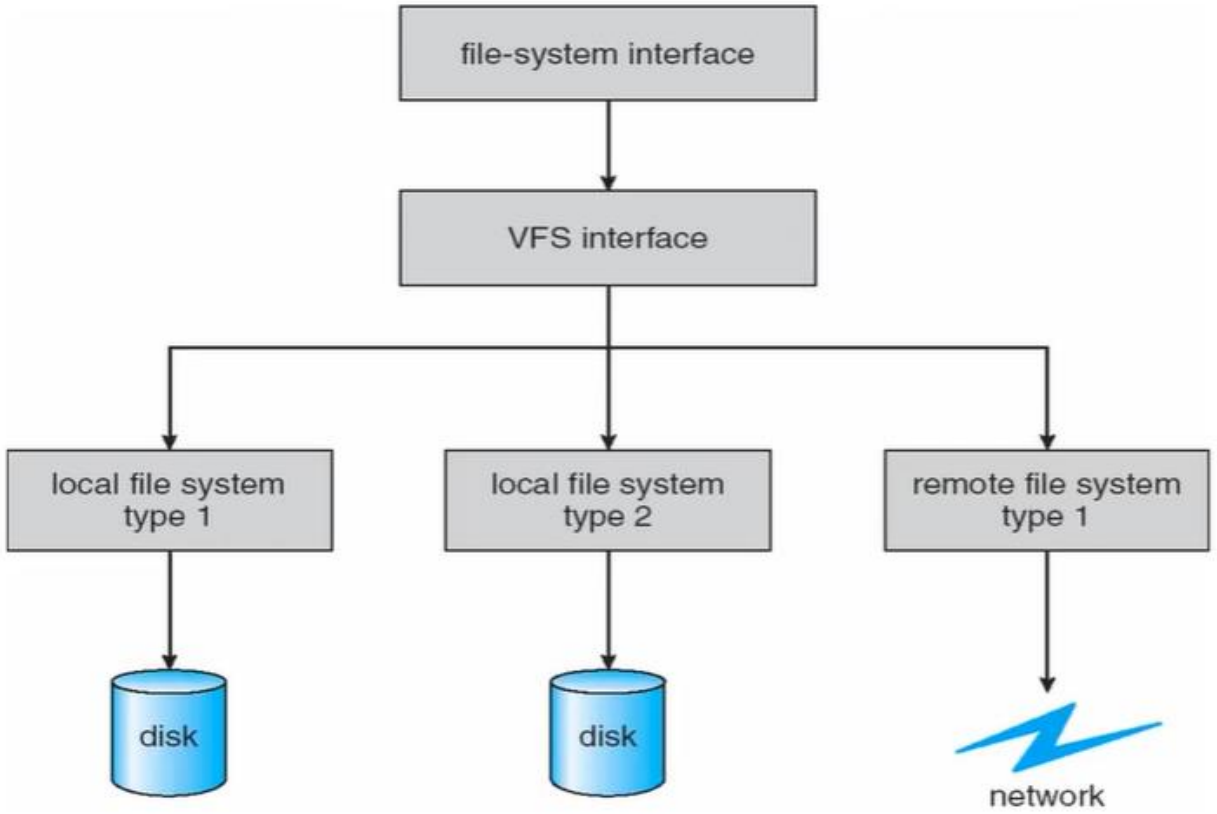
File Control Block

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

Virtual File System

- Virtual File Systems (VFS) on Unix provide an object-oriented way of implementing file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems
  - The API is to the VFS interface, rather than any specific type of file system

Virtual File System



Allocation Methods

How disc blocks are allocated to files.

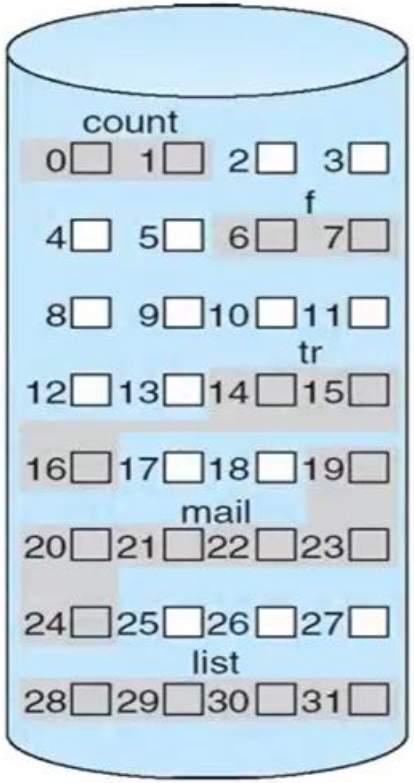
- Contiguous.
- Linked.
- Indexed.

AMRITA VISHWA VIDYAPEETHAM | AHEAD Online

Contiguous Allocation

- **Contiguous allocation** – each file occupies set of contiguous blocks
  - Best performance in most cases
  - Only starting location (block #) and length (number of blocks) are required
  - Problems include finding space for file, knowing file size, external fragmentation, need for compaction off-line (downtime) or on-line

Contiguous Allocation



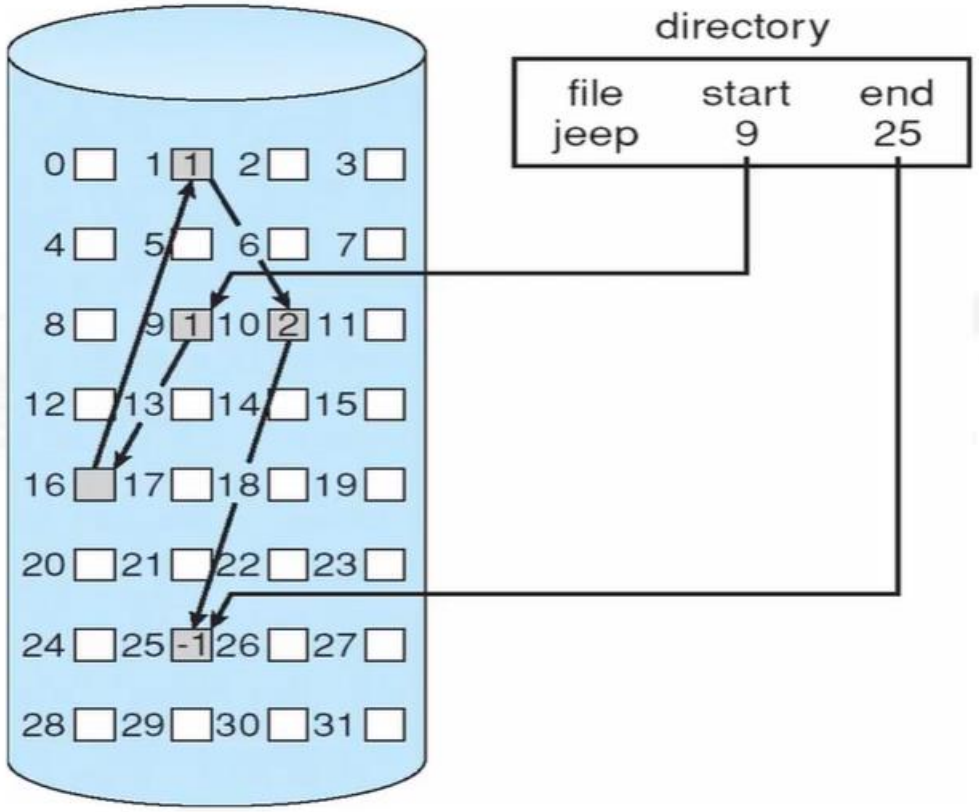
directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Linked Allocation

- **Linked allocation** –each file a linked list of blocks
  - File ends at nil pointer
  - No external fragmentation
  - Each block contains pointer to next block
  - No compaction, external fragmentation
  - Free space management system called when new block needed
  - Improve efficiency by clustering blocks into groups but increases internal fragmentation

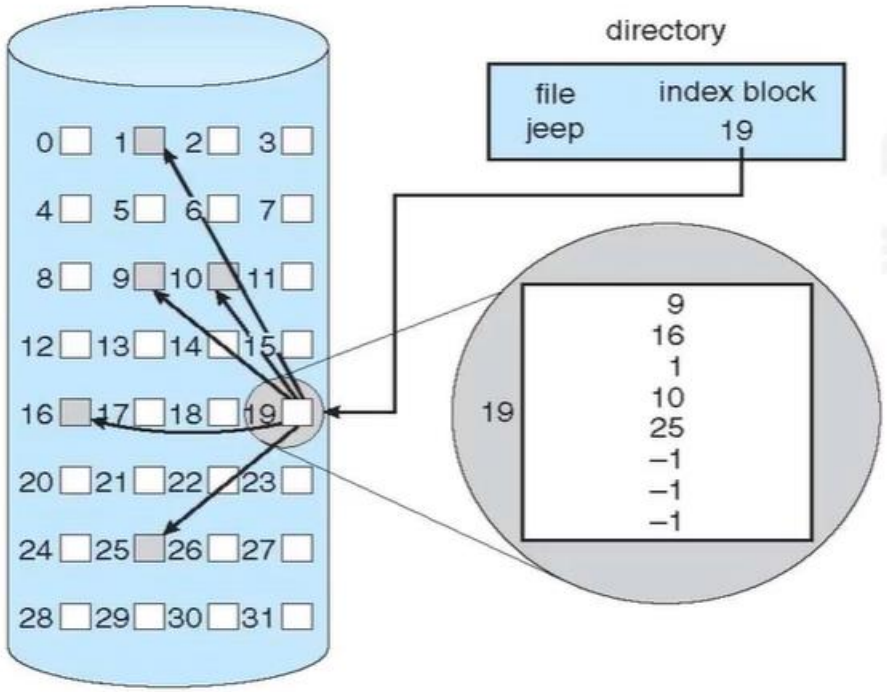
Linked Allocation



Indexed Allocation

- **Indexed allocation**
  - Each file has its own **index block**(s) of pointers to its data blocks

Indexed Allocation



Takeaways

- File system management.
- File control block.
- File allocation schemes.

References

1 Fundamentals Computers, V Rajaraman, Neeharika Adabala, Phi Learning Pvt. Ltd

2 J. Glenn Brookshear,"Computer Science: An Overview", Addision-Wesley, Twelth Edition, 2014

3 Computer Fundamentals, Anita Goel, Pearson